

DOCKER

Modül 17.3.5

Dockerfile Best Practices

Profesyoneller gibi Dockerfile yazmak

Data Science RoadMap

Alican Kaya

github.com/AlicanKaya192

Altın Kurallar (Best Practices)

1. Sadece Gerekli Olanları Base Alın

`ubuntu:latest` yerine her zaman `python:3.11-slim` veya `node:18-alpine` gibi minimal base imajları tercih edin. Küçük imajlar hızlı iner ve daha az güvenlik açığı (CVE) barındırır.

2. Katmanları Birleştirin (Layer Minimization)

Her RUN komutu yeni bir katman yaratır. Ardarda gelen `apt-get` veya komutlarını `&&` ile birleştirerek tek bir RUN talimatı haline getirin.

dockerfile

```
# KÖTÜ KULLANIM (3 ayrı katman)
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# İYİ KULLANIM (Tek katman)
RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*
```

3. Caching İçin Sıra Önemlidir (En az değişenden en çok değişene)

Gereksinim (requirements) kurulumu uzun sürer ancak nadir değişir. Uygulama kaynak kodu ise sürekli değişir. Bu yüzden önce bağımlılıkları kopyalayıp kurun, sonra kaynak kodunu kopyalayın.

İyi vs Kötü Dockerfile

[+] Mükemmel Dockerfile

- Minimal base image (alpine/slim)
- Kullanıcı yetkileri sınırlı (USER app)
- .dockerignore kullanılmış
- Bağımlılıklar (requirements) ayrı COPY edilmiş
- Gereksiz önbelgeler temizlenmiş (rm -rf...)

[-] Kötü Dockerfile

- Base image olarak tam işletim sistemi (ubuntu)
- Uygulama root yetkisiyle çalışıyor
- Tüm dosyalar COPY . ile atılmış (.git dahil)
- Gereksiz araçlar (vim, ping) yüklenmiş
- Tek bir satır değişse tüm paketler baştan iniyor

4. Root Kullanıcısından Kaçının

Konteynerler varsayılan olarak 'root' ile çalışır. Güvenlik için özel bir kullanıcı oluşturup USER talimatı ile ona geçiş yapın.

```
dockerfile
```

```
RUN groupadd -r appgroup && useradd -r -g appgroup appuser  
USER appuser
```

[i] NOT

Güvenlik açıklarını en aza indirmek için, konteyner içindeki uygulamanın sadece kendi işini yapacak kadar yetkiye sahip olması gerekir (Least Privilege).